

Laboratorio #2 — Redes Neuronales Convolucionales sobre MNIST

Ian Cumes · Carne 23236 · CC3092 Deep Learning y Sistemas Inteligentes · Universidad del Valle de Guatemala

1. Investigacion: capas de PyTorch para la CNN

Capa	Proposito	Parametros mas relevantes
<code>nn.Conv2d</code>	Aplica filtros aprendibles que se deslizan sobre la imagen y detectan patrones locales (bordes, esquinas, trazos). Es la capa que da a la CNN su invarianza a traslaciones: el mismo filtro se reutiliza en todas las posiciones.	<code>in_channels</code> , <code>out_channels</code> , <code>kernel_size</code> , <code>stride</code> , <code>padding</code> , <code>dilation</code> , <code>bias</code>
<code>nn.MaxPool2d</code>	Submuestra cada ventana quedandose con el valor maximo. Reduce la resolucion espacial, amplia el campo receptivo y conserva la activacion mas fuerte, por lo que preserva bien los bordes y trazos marcados. No tiene parametros entrenables.	<code>kernel_size</code> , <code>stride</code> , <code>padding</code> , <code>ceil_mode</code>
<code>nn.AvgPool2d</code>	Igual que <code>MaxPool2d</code> pero promediando la ventana: suaviza el mapa de activacion en vez de quedarse con el pico, conservando cuanta señal habia en la region y no solo su maximo. Cual de los dos conviene es empirico; aqui <code>AvgPool</code> resulto mejor (ver iteracion C3).	<code>kernel_size</code> , <code>stride</code> , <code>padding</code> , <code>count_include_pad</code>
<code>nn.BatchNorm2d</code>	Normaliza cada canal usando la media y varianza del batch y luego lo reescala con dos parametros aprendibles por canal. Estabiliza y acelera el entrenamiento, y actua como regularizador leve por el ruido del batch.	<code>num_features</code> , <code>eps</code> , <code>momentum</code> , <code>affine</code> , <code>track_running_stats</code>
<code>nn.Flatten</code>	Aplana el tensor (N, C, H, W) a (N, C*H*W) sin parametros ni computo real: es el puente entre la parte convolucional y el clasificador denso.	<code>start_dim</code> , <code>end_dim</code>
<code>nn.CrossEntropyLoss</code>	Perdida estandar de clasificacion multiclase. Combina <code>log_softmax</code> y <code>NLLoss</code> en una sola operacion numericamente estable, por lo que el modelo debe entregar logits crudos, sin softmax en la ultima capa.	<code>weight</code> , <code>ignore_index</code> , <code>reduction</code> , <code>label_smoothing</code>

Tensor. Es el arreglo multidimensional que PyTorch usa para todos los datos y parametros. Ademas de los valores guarda el `dtype`, el dispositivo (CPU/GPU) y, si `requires_grad=True`, el grafo de operaciones que permite calcular gradientes por autograd. En este laboratorio un batch de imagenes es un tensor (N, 1, 28, 28) y el MLP lo convierte en (N, 784).

Campo receptivo. Es la region de la imagen de entrada que influye en una sola activacion de una capa profunda. Crece de forma incremental: $RF_{out} = RF_{in} + (k - 1) \cdot jump$, donde `jump` es el producto de los strides acumulados. En la CNN ganadora (2 bloques conv 3x3 con pooling 2x2) el campo receptivo final es de **10x10 pixeles**, es decir 36% del ancho de la imagen: la primera capa solo ve trazos de 3x3 y cada bloque adicional amplia esa ventana, de modo que apilar capas hace crecer el contexto sin agrandar el kernel.

Por que la CNN necesita menos parametros. Por dos mecanismos. (1) *Conectividad local*: cada neurona solo se conecta a una ventana de kxk pixeles, no a los 784. (2) *Pesos compartidos*: el mismo filtro se reutiliza en todas las posiciones de la imagen, asi que su costo no escala con la resolucion. Un `Conv2d(1, 32, 3)` tiene solo 320 parametros y produce 32 mapas de 28x28, mientras que una capa densa `Linear(784, 32)` necesita 25 120 parametros y ademas destruye la estructura espacial al aplanar.

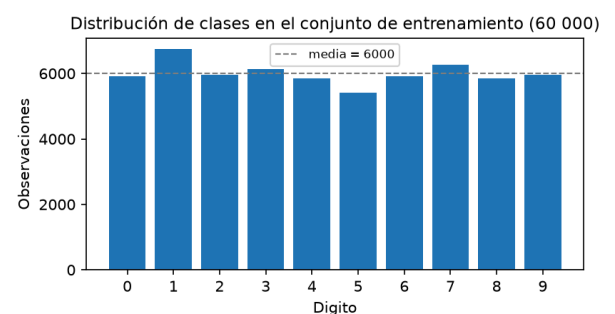
Matiz importante que muestran los resultados. El ahorro es real *por capa*, pero no se traduce automaticamente en un modelo total mas chico: la CNN ganadora usa 421 642 parametros, 1.8x mas que el MLP. La razon aparece al desglosarla: solo **18 816 (4.5%)** estan en los bloques convolucionales y **402 826 (95.5%)** en la cabeza densa posterior al `Flatten`. Es decir: la parte convolucional es baratisima y casi todo el costo lo pone el clasificador denso, que es justamente el componente de tipo MLP. La eficiencia en parametros si se ve al comparar iteraciones: **C1** (206 922 parametros) supera al mejor MLP **M6** (235 914) en F1 de validacion — 98.78% contra 98.24% — usando **menos** parametros que el.

2. Datos y protocolo experimental

Dataset. MNIST: **70 000** imagenes de digitos manuscritos (60 000 de entrenamiento + 10 000 de test) en **10 clases** (0-9). Cada imagen es de **28x28 pixeles** en escala de grises, con valores enteros en [0, 255] que `ToTensor()` escala a [0, 1]. Las clases estan *aproximadamente* balanceadas: van del ~9.0% (digito 5) al ~11.2% (digito 1), razon max/min ≈ 1.24. Ese desbalance leve no exige remuestreo, pero si justifica reportar metricas **macro** y estratificar el split.

Normalizacion. Necesaria. Ademas del escalado a [0, 1] se estandariza con `Normalize(μ=0.1306, σ=0.3081)`: como ~80% de los pixeles son fondo negro, sin centrar la entrada media queda lejos de 0 y el descenso de gradiente zigzaguea. Ambos estadisticos se calculan **solo sobre el split de entrenamiento** para no filtrar informacion de validacion ni de test hacia el preprocesamiento.

Particion y protocolo. 54 000 entrenamiento / 6 000 validacion (estratificado, `random_state=23236`) + 10 000 test. Toda la seleccion de hiperparametros se hace mirando **solo validacion**; el conjunto de test se evalua **una unica vez** al final, con la mejor configuracion ya congelada. Semilla 23236 fijada en `random`, `numpy` y `torch` al inicio de cada iteracion, de modo que las diferencias entre iteraciones sean atribuibles al hiperparametro y no a la inicializacion.

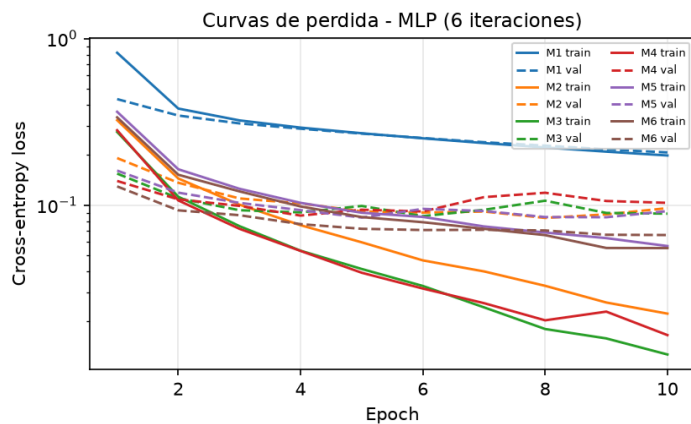


Distribucion de clases en entrenamiento: desbalance leve (~9%-11%)

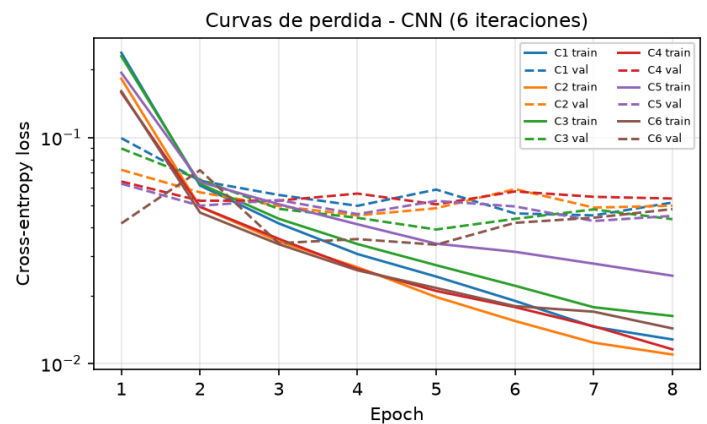
3. Resultados de las iteraciones (12 iteraciones: 6 MLP + 6 CNN)

Busqueda sistematica cambiando **una variable a la vez** respecto de la iteracion indicada en la columna **Base**, de modo que cada delta de metrica sea atribuible a un unico cambio. La busqueda es un arbol y no una cadena: C3 y C4 son dos ramas de C2, para no mezclar el cambio de pooling con el de normalizacion. Las metricas son **macro** sobre el conjunto de **validacion** (6 000 imagenes); el conjunto de test no se toco durante la busqueda.

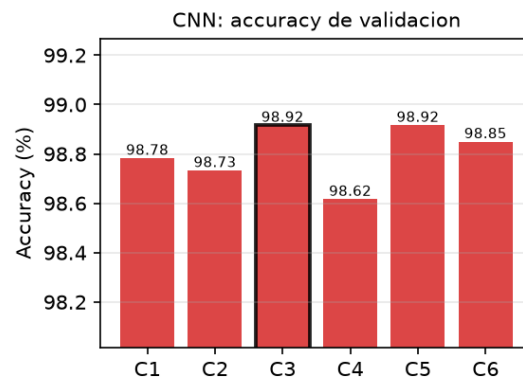
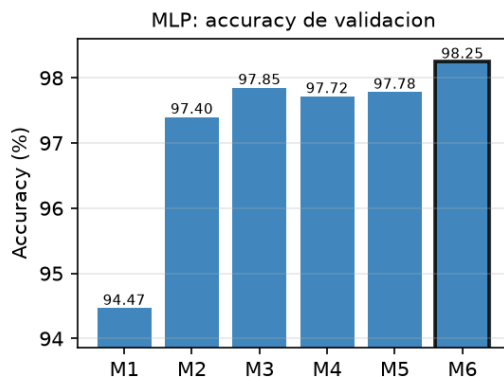
ID	Base	Arq.	Cambio respecto a su base	Configuracion	Epochs	Train loss	Val loss	Accuracy	Precision	Recall	F1	Params	Tiempo (s)
M1	—	MLP	Baseline: 1 capa oculta de 128, SGD lr=0.01	128, sgd lr=0.01	10	0.1987	0.2072	94.47%	94.43%	94.44%	94.43%	101,770	89.5
M2	M1	MLP	Optimizador SGD -> Adam (lr=1e-3)	128, adam lr=0.001	10	0.0223	0.0959	97.40%	97.40%	97.38%	97.37%	101,770	93.6
M3	M2	MLP	Ancho de la capa oculta 128 -> 256	256, adam lr=0.001	10	0.0127	0.0889	97.85%	97.85%	97.83%	97.84%	203,530	111.3
M4	M3	MLP	Profundidad: 1 -> 2 capas ocultas (256, 128)	256-128, adam lr=0.001	10	0.0166	0.1032	97.72%	97.75%	97.66%	97.69%	235,146	140.0
M5	M4	MLP	Regularizacion: + Dropout 0.3	256-128, adam lr=0.001, drop=0.3	10	0.0568	0.0921	97.78%	97.79%	97.78%	97.77%	235,146	126.1
M6	M5	MLP	Normalizacion: + BatchNorm1d	256-128, adam lr=0.001, BN, drop=0.3	10	0.0552	0.0662	98.25%	98.23%	98.25%	98.24%	235,914	137.8
C1	—	CNN	Baseline: 2 bloques conv (16, 32), MaxPool, Adam lr=1e-3	16-32, pool=max, fc=128, adam lr=0.001	8	0.0128	0.0516	98.78%	98.77%	98.79%	98.78%	206,922	149.5
C2	C1	CNN	Capacidad: canales (16,32) -> (32,64)	32-64, pool=max, fc=128, adam lr=0.001	8	0.0110	0.0499	98.73%	98.73%	98.74%	98.73%	421,642	230.2
C3	C2	CNN	Pooling: MaxPool2d -> AvgPool2d	32-64, pool=avg, fc=128, adam lr=0.001	8	0.0162	0.0435	98.92%	98.93%	98.92%	98.92%	421,642	229.7
C4	C2	CNN	Normalizacion: + BatchNorm2d	32-64, pool=max, fc=128, adam lr=0.001, BN	8	0.0116	0.0537	98.62%	98.61%	98.64%	98.62%	421,834	266.3
C5	C4	CNN	Regularizacion: + Dropout 0.25 antes de la FC final	32-64, pool=max, fc=128, adam lr=0.001, BN, drop=0.25	8	0.0245	0.0450	98.92%	98.92%	98.92%	98.92%	421,834	264.5
C6	C5	CNN	Profundidad: 2 -> 3 bloques conv (32, 64, 128)	32-64-128, pool=max, fc=128, adam lr=0.001, BN, drop=0.25	8	0.0143	0.0483	98.85%	98.87%	98.84%	98.86%	241,994	285.6



Curvas de perdida — MLP (6 iteraciones; continua = train, punteada = val)



Curvas de perdida — CNN (6 iteraciones; continua = train, punteada = val)

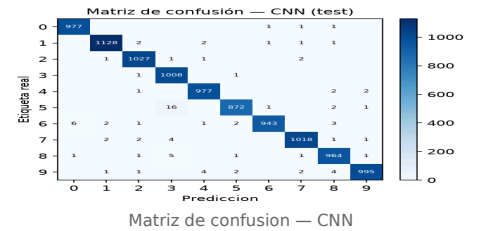
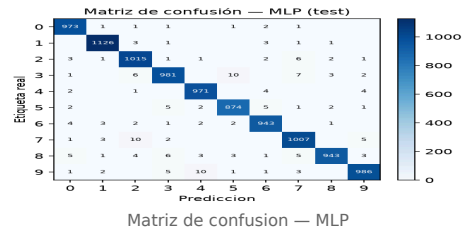
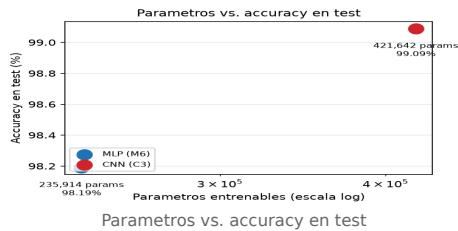


Accuracy de validacion por iteracion; el borde negro marca la mejor de cada arquitectura

4. Comparacion de arquitecturas (conjunto de test, una sola evaluacion)

Mejor configuracion de cada arquitectura segun F1-macro de validacion: **MLP M6** (256-128, adam lr=0.001, BN, drop=0.3) y **CNN C3** (32-64, pool=avg, fc=128, adam lr=0.001).

Arquitectura	Mejor iteracion	Parametros entrenables	Accuracy (test)	Precision (test)	Recall (test)	F1 (test)	Tiempo entren. (s)	Inferencia (ms/img)
MLP	M6	235,914	98.19%	98.19%	98.18%	98.18%	200.6	0.081
CNN	C3	421,642	99.09%	99.09%	99.07%	99.08%	295.7	0.327



5. Discusion y analisis

1. Mayor impacto positivo y negativo. En el MLP el cambio mas beneficioso fue *Optimizador SGD -> Adam (lr=1e-3)* (+0.0295 de F1 en validacion, iteracion M2), y el mas perjudicial *Profundidad: 1 -> 2 capas ocultas (256, 128)* (-0.0015). En la CNN el mayor salto vino de *Regularizacion: + Dropout 0.25 antes de la FC final* (+0.0030) y el mayor retroceso de *Normalizacion: + BatchNorm2d* (-0.0011). El patron general es que los cambios de *optimizacion* y de *arquitectura* mueven mucho mas la aguja que los ajustes finos de capacidad, y que a partir de cierto punto agregar parametros ya no compra accuracy.

2. Overfitting / underfitting. Se identifico leyendo la brecha entre la perdida de entrenamiento y la de validacion en las curvas: cuando la de entrenamiento sigue bajando y la de validacion se estanca o sube, el modelo esta memorizando. La iteracion con mayor brecha fue **M4** (MLP), con val-train = +0.0866. El underfitting aparecio en la baseline del MLP (M1, SGD sin momentum), donde ambas perdidas se quedaban altas y planas: ahi el problema no era capacidad sino velocidad de convergencia. Se mitigo con Dropout y BatchNorm en las iteraciones posteriores y limitando el numero de epochs.

3. Efecto de la regularizacion. En el MLP, + BatchNorm1d mejoro el F1 de validacion en +0.0046. En el MLP, + Dropout 0.3 mejoro el F1 de validacion en +0.0008. En la CNN, + Dropout 0.25 antes de la FC final mejoro el F1 de validacion en +0.0030. En la CNN, + BatchNorm2d empeoro el F1 de validacion en -0.0011. Ningun metodo gana en ambas arquitecturas: BatchNorm ayudo al MLP pero perjudico a la CNN, y Dropout hizo lo contrario. La regularizacion rinde en proporcion al overfitting que hay que corregir, y con 54 000 ejemplos limpios ese margen es estrecho: los deltas de la CNN son de milésimas, dentro del ruido de una sola semilla, asi que lo honesto es elegir por arquitectura y no declarar un ganador general.

4. MLP vs CNN en test. Gano la **CNN**: 99.09% de accuracy frente a 98.19% del MLP (0.90 puntos porcentuales, equivalente a reducir el error en 49.7%), con 1.8x mas parametros (421 642 vs 235 914). Conviene no leer esa relacion al reves: la ventaja **no** viene de tener mas capacidad. La prueba es que C1, con 206 922 parametros, ya supera al mejor MLP usando *menos* parametros que el; y que dentro de las propias CNN aumentar canales de (16,32) a (32,64) *empeora* el F1. La razon es estructural: el MLP aplanar la imagen y trata cada pixel como una feature independiente, de modo que pierde toda la informacion de vecindad y debe reaprender el mismo trazo en cada posicion. La CNN preserva la estructura 2D, comparte los filtros entre posiciones y construye el campo receptivo por etapas, asi que la invarianza a pequenas traslaciones le sale "gratis" en vez de tener que aprenderla con mas pesos.

5. Tipos de error. Las confusiones dominantes del MLP fueron 3→5 (10), 7→2 (10), 9→4 (10) y las de la CNN 5→3 (16), 6→0 (6), 8→3 (5). Ambos modelos se equivocan sobre todo entre digitos que comparten trazos (4/9, 3/5, 7/2), pero el MLP acumula ademas errores en pares que solo se distinguen por la *posicion relativa* de los trazos, precisamente lo que se pierde al aplanar; la CNN concentra sus pocos fallos en digitos escritos de forma genuinamente ambigua.

6. Modelo para produccion. Elegiria una **CNN**, pero reconociendo que aqui si hay un trade-off real y no fingiendo que la CNN domina en todo. La CNN ganadora es mas exacta (99.09% vs 98.19%) pero tambien mas pesada (421 642 parametros, 1.8x mas que el MLP) y mas lenta en inferencia (0.327 ms/imagen en CPU contra 0.081 ms del MLP, es decir 4.0x). Aun asi la eleccion es clara: 0.90 puntos porcentuales equivalen a 50% menos errores, y a escala de produccion cada error mal clasificado cuesta intervencion manual, que es ordenes de magnitud mas cara que 0.327 ms de CPU. Si la memoria fuera la restriccion vinculante, la configuracion a desplegar seria C1: 206 922 parametros, menos que el MLP, y aun por encima de el en validacion. El MLP solo ganaria en un dispositivo incapaz de ejecutar convoluciones de forma eficiente.

Conclusiones

- La CNN alcanzo **99.09%** de accuracy en test contra **98.19%** del MLP: 50% menos errores.
- El numero de parametros no predice la calidad: la CNN ganadora tiene 1.8x mas parametros que el MLP, pero C1 lo supera con *menos*, y subir canales de (16,32) a (32,64) empeora el resultado. Lo que decide es si la arquitectura respeta la estructura del dato, no cuanta capacidad tiene.
- El costo en parametros de una CNN no esta en las convoluciones (4.5% del total) sino en la cabeza densa (95.5%): para achicar el modelo hay que atacar el clasificador, no los filtros.
- Varias intuiciones estandar no sobrevivieron a la medicion: AvgPool2d supero a MaxPool2d, mas canales empeoraron el resultado y BatchNorm2d perjudico a la CNN. Medir una variable a la vez fue lo que permitio detectarlo y atribuir cada cambio a su causa.

Repositorio: <https://github.com/iancumes/Lab2DeepLearning>

Notebook completo y comentado: notebooks/Lab2_CNN_MNIST_IanCumes_23236.ipynb